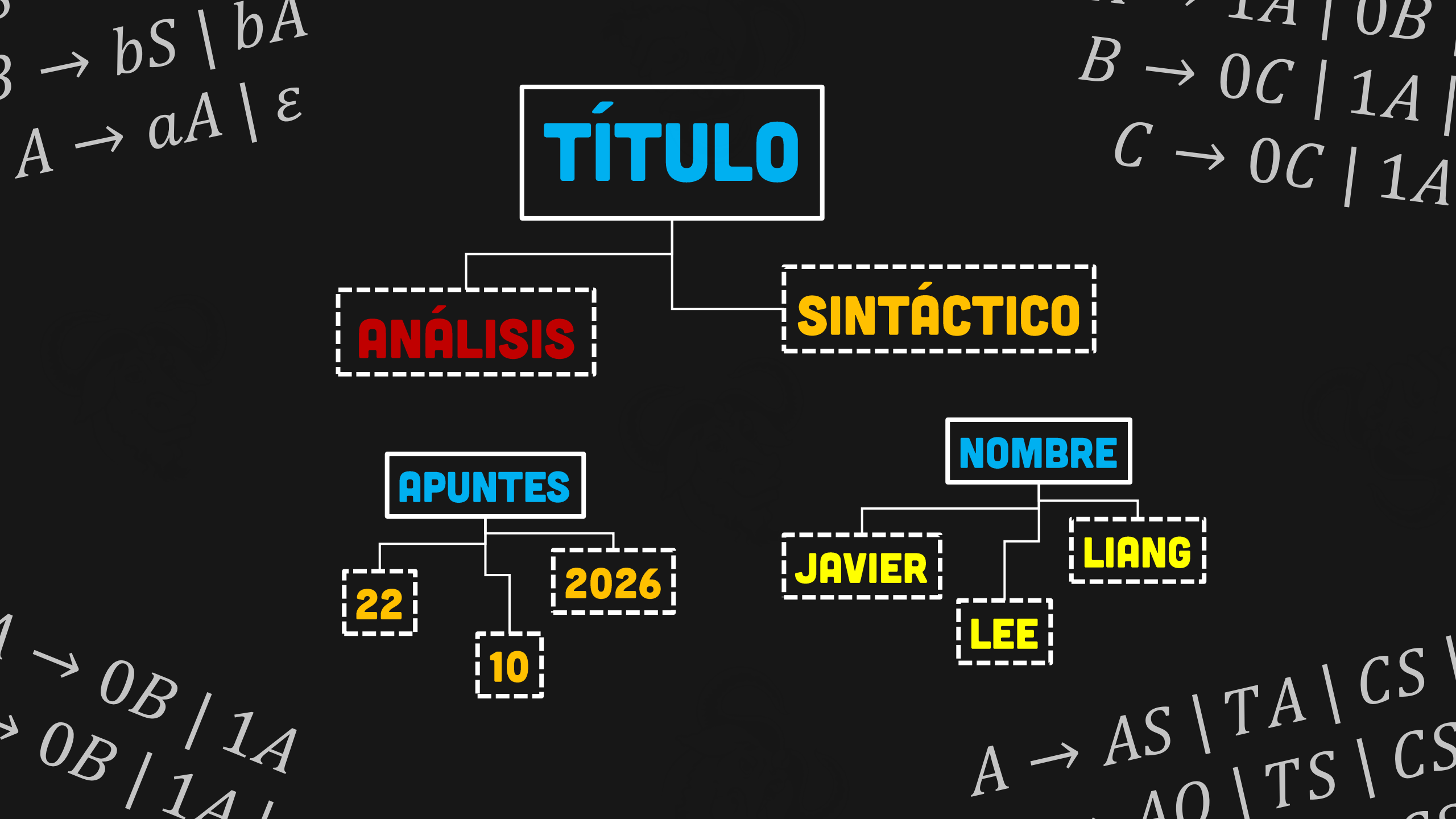
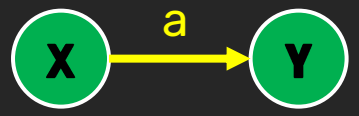

TÍTULO
ANÁLISIS
SINTÁCTICO
APUNTES
22
10
2026
NOMBRE
JAVIER
LEE
LIANG

REPASO DE LA CLASE PASADA

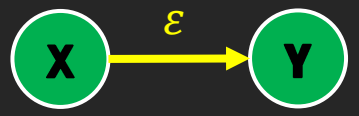
Algoritmo de un DFA, NFA o NFA-ε a una Gramática Regular

1) A cada estado del autómata le corresponde a un No Terminal. El Estado Inicial será el símbolo inicial

- Para cada tipo de transición, agregue una regla de la siguiente forma:

2) 

$\Longrightarrow \quad X \to aY$

3) 

$\Longrightarrow \quad X \to Y$

4) 

$\Longrightarrow \quad X \to \varepsilon$

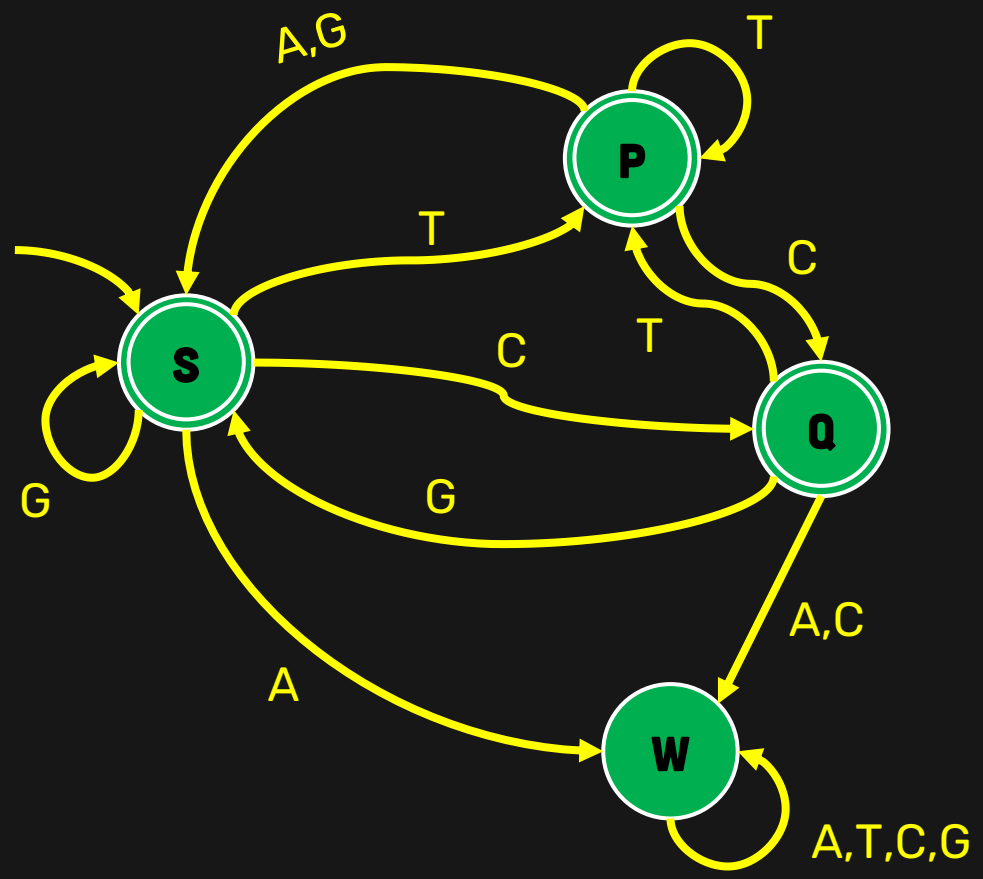
5) Para los sink, simplemente ignorarlos, no se convierte en ninguna regla

VIMOS UNOS CUANTOS EJEMPLOS EN LA CLASE ANTERIOR

PERO DE LOS EJEMPLOS 1 AL 4 ESTÁN EN LOS APUNTES DEL MARTES DE ESTA SEMANA

EJEMPLO 5

Sea $\mathcal{L}$ el lenguaje sobre Σ = {A, T, C, G} de hileras que no contengan la subhilera “CC”



Regla 1, 2, 4: $S \rightarrow TP \mid CQ \mid GS \mid \varepsilon$

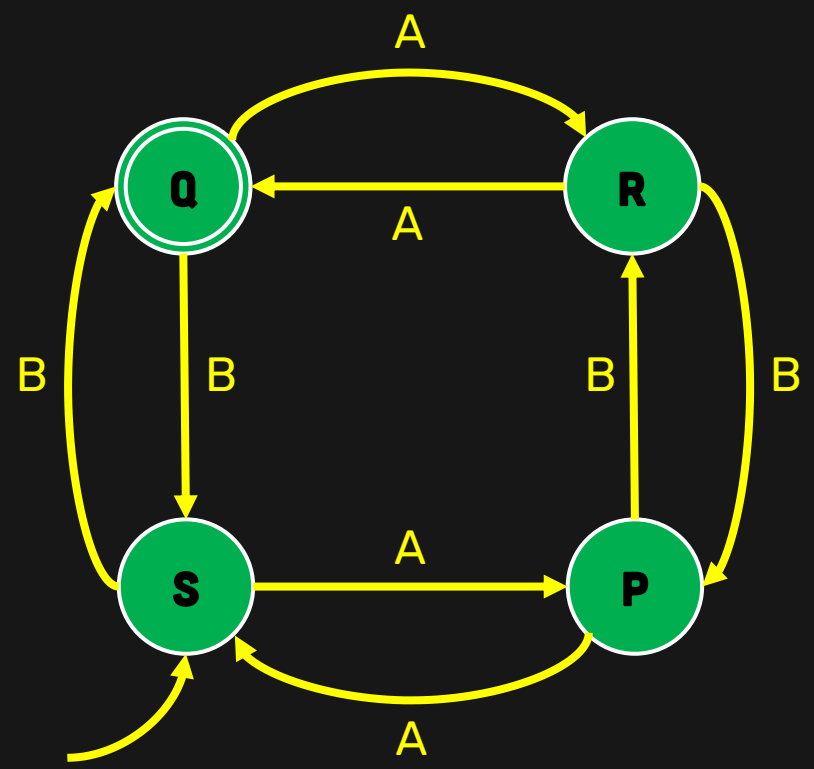
Regla 2, 4: $P \rightarrow AS \mid TP \mid CQ \mid GS \mid \varepsilon$

Regla 2, 4: $\mathrm{Q} \rightarrow TP \mid GS \mid \varepsilon$

Regla 5: Omitir el estado **W**

EJEMPLO 6

Sea $\mathcal{L}$ el lenguaje sobre Σ = {A, B} que contengan un número par de A's y un número Impar de B's



Regla 1, 2: $S \rightarrow AP \mid BQ$

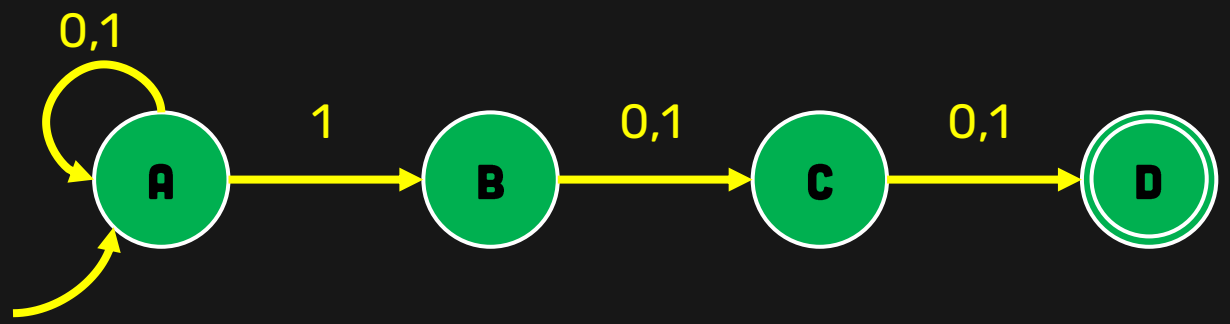
Regla 2: $P \rightarrow AS \mid BR$

Regla 2: $R \rightarrow AQ \mid BP$

Regla 2, 4: $\mathrm{Q} \rightarrow AQ \mid BP \mid \varepsilon$

EJEMPLO 7

Presente una gramática regular equivalente al siguiente NFA



Regla 1, 2: $A \rightarrow 0A \mid 1A \mid 1B$

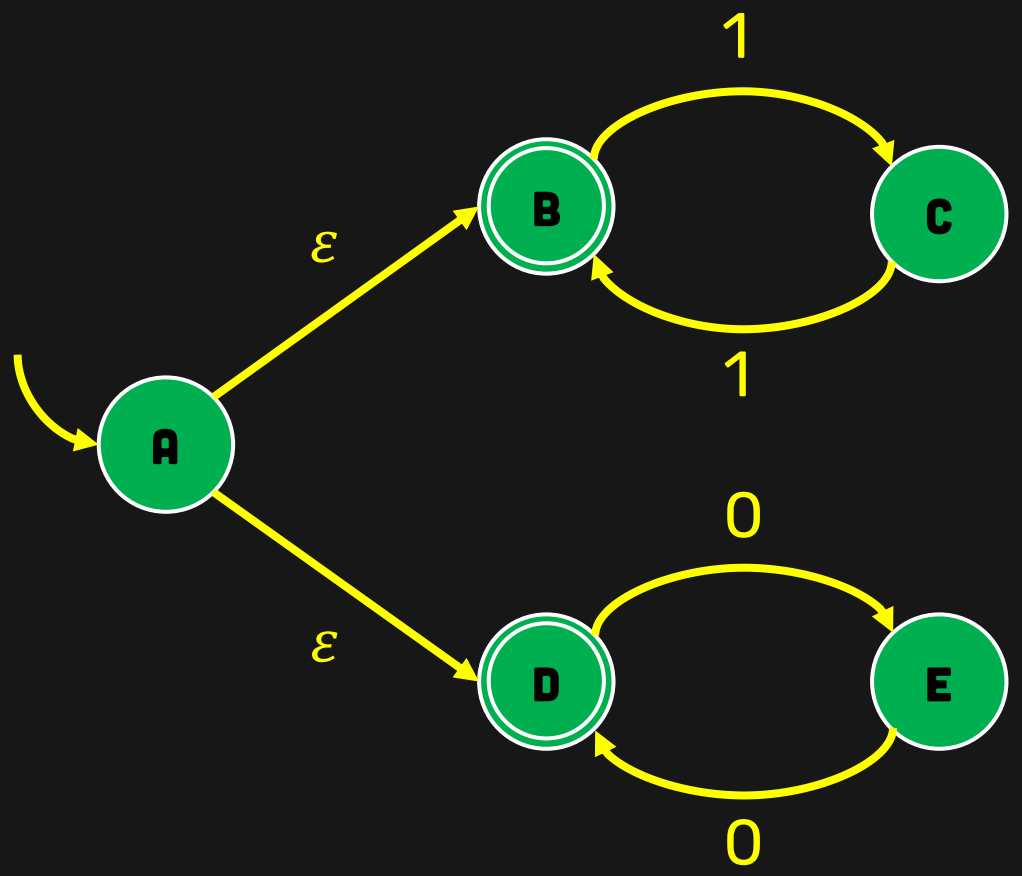
Regla 2: $B \rightarrow 0C \mid 1C$

Regla 2: $C \rightarrow 0D \mid 1D$

Regla 4: $D \rightarrow \varepsilon$

EJEMPLO 8

PRESENTE UNA GRÁMATICA REGULAR EQUIVALENTE A ESTE NFA-ε



Regla 1, 3: $A \rightarrow B \mid D$

Regla 2, 4: $B \rightarrow 1C \mid \varepsilon$

Regla 2: $C \rightarrow 1B$

Regla 2, 4: $D \rightarrow 0E \mid \varepsilon$

Regla 2: $E \rightarrow 0D$

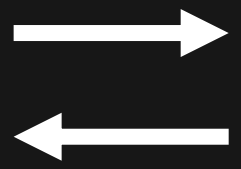
LENGUAJES LIBRE DE CONTEXTO

- Definición

  Un lenguaje L es un lenguaje libre de contexto (CNL) si es completamente generado por una CFG, con G = (V, Σ, P, S), además G no genera hileras que no pertenezcan al lenguaje.
- Entonces si nos dicen que un lenguaje es libre de contexto, significa que existe un CFG que lo genera y viceversa

- ¿Cómo se demuestra que un lenguaje es un CFL
  - Diseñando una CFG que lo genera
- ¿Cómo se demuestra que un lenguaje no es CFL?
  - Demostrando que no existe ninguna CFG que lo genera
  - Pumping Lemma para CFL

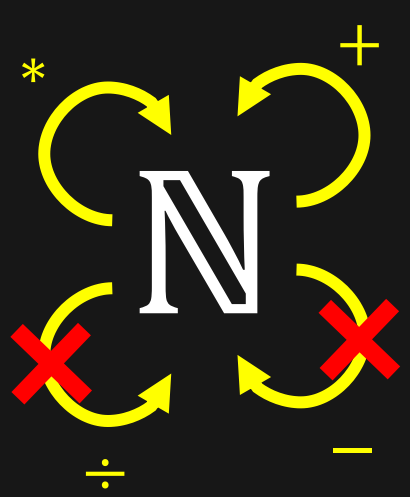
$\mathcal{L}$ es libre de contexto

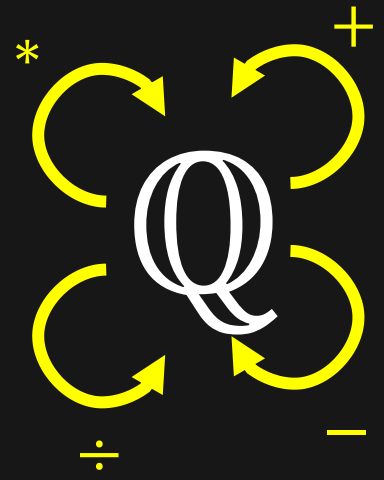


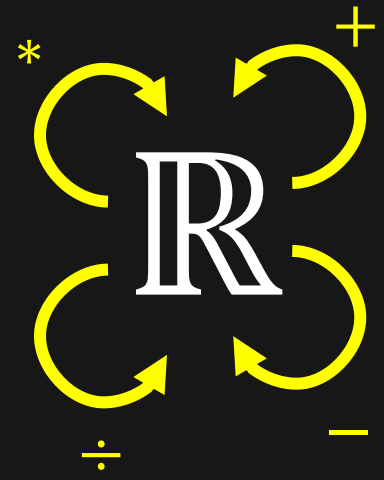
$S \to \cdots \mid \cdots$
$T \to \cdots \mid \cdots$
$U \to \cdots \mid \cdots$
$V \to \cdots \mid \cdots$

REPASO DE CIERRE EN CONJUNTOS

- Un conjunto X puede ser “Cerrado con respecto a una operación φ”
- Significa que siempre que aplicamos la operación φ sobre elementos de X, obtenemos algo que también es elemento de X







CIERRE EN UNIÓN DE CFL

- Teorema

  Si $\mathcal{L}$ y $\mathcal{M}$ son CFL entonces $\mathcal{L} \cup \mathcal{M}$ también es CFL
- Los CFL son cerrados con respecto a la unión de lenguajes

PREMISAS

- Si $\mathcal{L} \cup \mathcal{M}$ fuera CFL, existiría una CFG que lo genere

$\mathcal{L}$ es un CFL $\longrightarrow$ $S_1 \rightarrow \cdots \mid \cdots$, $A \rightarrow \cdots \mid \cdots$

$\mathcal{M}$ es un CFL $\longrightarrow$ $S_2 \rightarrow \cdots \mid \cdots$, $B \rightarrow \cdots \mid \cdots$

DEMOSTRACIÓN

- Sin pérdida de generalidad, supongamos que los No Terminales de cada CFG los cambiamos para que sean diferentes

$\longrightarrow$

$S \rightarrow S_1 \mid S_2$
$S_1 \rightarrow \cdots \mid \cdots$
$A \rightarrow \cdots \mid \cdots$
$S_2 \rightarrow \cdots \mid \cdots$
$B \rightarrow \cdots \mid \cdots$

$\longrightarrow$ $\mathcal{L} \cup \mathcal{M}$ es un CFG

EJEMPLO DE UNIÓN DE CFL

$\mathcal{L}$

$\{a^n b^m c^m d^{2n}, n \geq 0, m > 0\}$

$S \rightarrow aSdd \mid A$
$A \rightarrow bAc \mid bc$

$\mathcal{M}$

$\{a^n b^m, n \geq 0, m \geq 0\}$

$S \rightarrow AB$
$A \rightarrow aA \mid \varepsilon$
$B \rightarrow bB \mid \varepsilon$

U

$\mathcal{L} \cup \mathcal{M}$

$S \rightarrow S_1 \mid S_2$
$S_1 \rightarrow aSdd \mid A$
$A \rightarrow bAc \mid bc$
$S_2 \rightarrow AB$
$A \rightarrow aA \mid \varepsilon$
$B \rightarrow bB \mid \varepsilon$

CIERRE EN CONCATENACIÓN DE CFL

- Teorema

  Si $\mathcal{L}$ y $\mathcal{M}$ son CFL entonces

  $\mathcal{LM}$ también es CFL
- Los CFL son cerrados con respecto a la concatenación de lenguajes

PREMISAS

- Si $\mathcal{LM}$ fuera CFL, existiría una CFG que lo genere

$\mathcal{L}$ es un CFL $\longrightarrow$ $S_1 \to \cdots \mid \cdots$, $A \to \cdots \mid \cdots$

$\mathcal{M}$ es un CFL $\longrightarrow$ $S_2 \to \cdots \mid \cdots$, $B \to \cdots \mid \cdots$

DEMOSTRACIÓN

- Sin pérdida de generalidad, supongamos que los No Terminales de cada CFG los cambiamos para que sean diferentes

$\longrightarrow$

$$S \to S_1 S_2$$
$$S_1 \to \cdots \mid \cdots$$
$$A \to \cdots \mid \cdots$$
$$S_2 \to \cdots \mid \cdots$$
$$B \to \cdots \mid \cdots$$

$\longrightarrow$ $\mathcal{LM}$ es un CFG

EJEMPLO DE CONCATENACIÓN DE CFL

$\mathcal{L}$

$\{a^n b^m c^m d^{2n}, n \geq 0, m > 0\}$

$S \rightarrow aSdd \mid A$
$A \rightarrow bAc \mid bc$

$\mathcal{M}$

$\{a^n b^m, n \geq 0, m \geq 0\}$

$S \rightarrow AB$
$A \rightarrow aA \mid \varepsilon$
B$\rightarrow bB \mid \varepsilon$

concatenar

$\mathcal{LM}$

$S \rightarrow S_1 S_2$
$S_1 \rightarrow aSdd \mid A$
$A \rightarrow bAc \mid bc$
$S_2 \rightarrow AB$
$A \rightarrow aA \mid \varepsilon$
B$\rightarrow bB \mid \varepsilon$

CIERRE DE KLEENE DE CFL

- Teorema

  Si $\mathcal{L}$ y $\mathcal{M}$ son CFL entonces

  $\mathcal{L}^*$ también es CFL
- Los CFL son cerrados con respecto al cierre de Kleene de lenguajes

PREMISAS

- Si $\mathcal{L}^*$ fuera CFL, existiría una CFG que lo genere

$\mathcal{L}$ es un CFL $\longrightarrow$ $S_1 \to \cdots \mid \cdots$, $A \to \cdots \mid \cdots$

DEMOSTRACIÓN

- Agregamos un nuevo No Terminal y lo usamos como símbolo inicial

$\longrightarrow$ $S \to S_1 S \mid \varepsilon$, $S_1 \to \cdots \mid \cdots$, $A \to \cdots \mid \cdots$ $\longrightarrow$ $\mathcal{L}^*$ es un CFG

EJEMPLO DE CIERRE DE KLEENE DE CFL

$\mathcal{L}$

$\{a^n b^m c^m d^{2n}, n \geq 0, m > 0\}$

$S \rightarrow aSdd \mid A$
$A \rightarrow bAc \mid bc$



$\mathcal{L}^*$

$R \rightarrow SR \mid \varepsilon$
$S \rightarrow aSdd \mid A$
$A \rightarrow bAc \mid bc$

CIERRE CRUZ DE CFL

- Teorema

  Si $\mathcal{L}$ y $\mathcal{M}$ son CFL entonces

  $\mathcal{L}^+$ también es CFL
- Los CFL son cerrados con respecto al cierre de cruz de lenguajes

PREMISAS

- Si $\mathcal{L}^+$ fuera CFL, existiría una CFG que lo genere

$\mathcal{L}$ es un CFL $\longrightarrow$ $S_1 \to \cdots \mid \cdots$, $A \to \cdots \mid \cdots$

DEMOSTRACIÓN

- Agregamos un nuevo No Terminal y lo usamos como símbolo inicial

$\longrightarrow$ $S \to S_1 S \mid S_1$, $S_1 \to \cdots \mid \cdots$, $A \to \cdots \mid \cdots$ $\longrightarrow$ $\mathcal{L}^+$ es un CFG

EJEMPLO DE CIERRE CRUZ DE CFL

$\mathcal{L}$

$\{a^n b^m c^m d^{2n}, n \geq 0, m > 0\}$

$S \rightarrow aSdd \mid A$
$A \rightarrow bAc \mid bc$



$\mathcal{L}^+$

$R \rightarrow SR \mid S$
$S \rightarrow aSdd \mid A$
$A \rightarrow bAc \mid bc$

RESUMEN DE LOS CIERRES EN CFL

- Teorema:

  Los CFLs son cerrados con respecto a:

  - Unión
  - Concatenación
  - Reverso
  - Cierre de Kleene
  - Cierre cruz
  - Otros...

  No son cerrados con respecto a:

  - Intersección
  - Complemento
  - Diferencia

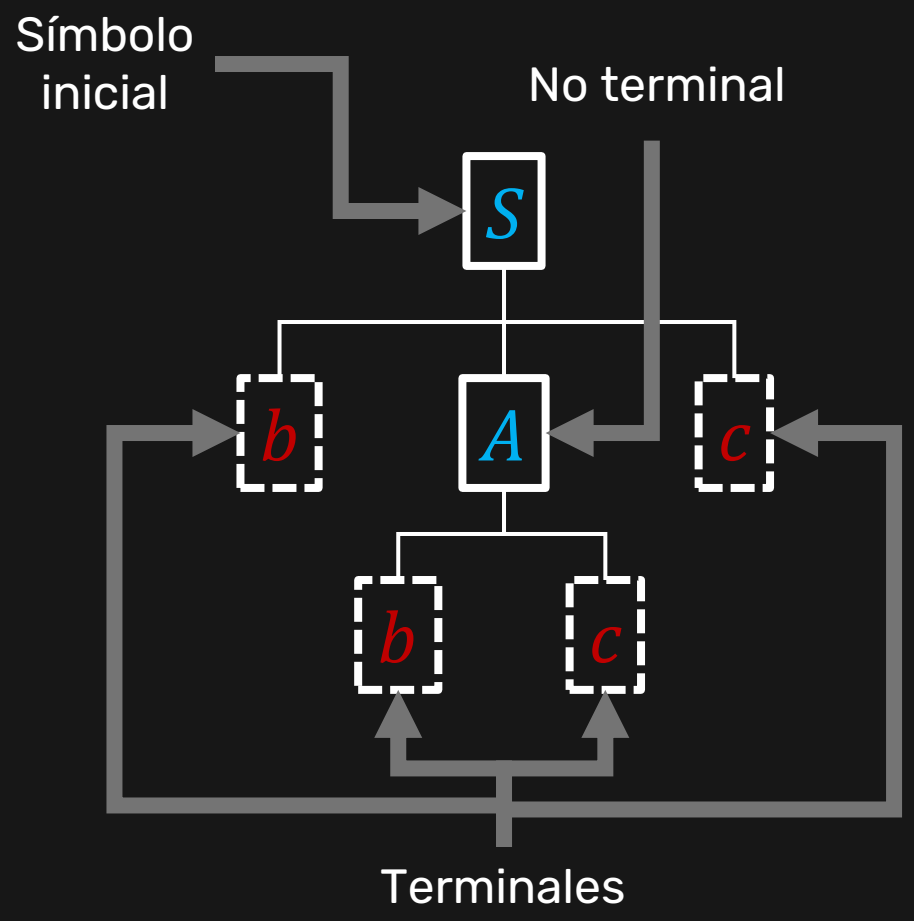
SE VIENE UN GRAN TEMA

YA NOS ESTAMOS ACERCANDO AL PARSER

ÁRBOLES DEDERIVACIÓN

ÁRBOLES Y DERIVACIÓN

- Una derivación se puede representar como un árbol de derivación
- Las hojas son terminales, los nodos intermedios son no terminales
- El símbolo inicial de la gramática es la raíz del árbol
- Cada vez que se expande un terminal con una regla, sus cmponentes pasan a ser los hojos de este nodo
- Si se lee las hojas de izquierda a derecha, se tiene la hilera derivada

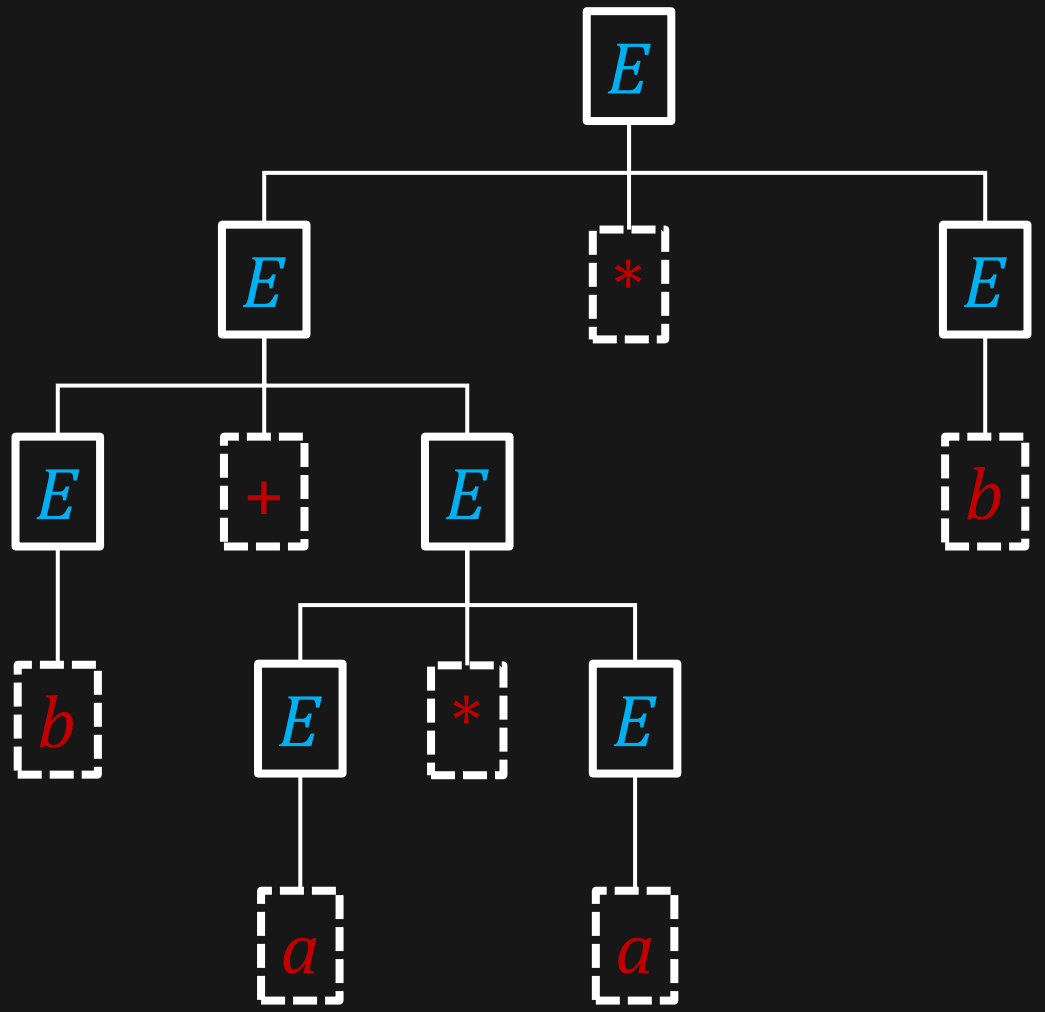


EJEMPLO 1

- Sea G definada como:

$$E \to E + E$$
$$E \to E * E$$
$$E \to a$$
$$E \to b$$

- Posible derivación:
  - E
  - $\Longrightarrow E * E$
  - $\Longrightarrow E + E * E$
  - $\Longrightarrow b + E * E$
  - $\Longrightarrow b + E * E * E$
  - $\Longrightarrow b + a * E * E$
  - $\Longrightarrow b + a * a * E$
  - $\Longrightarrow b + a * a * b$



EJEMPLO 2

- Considere la CFG:

  $Exp \rightarrow Exp\ Op\ Exp$

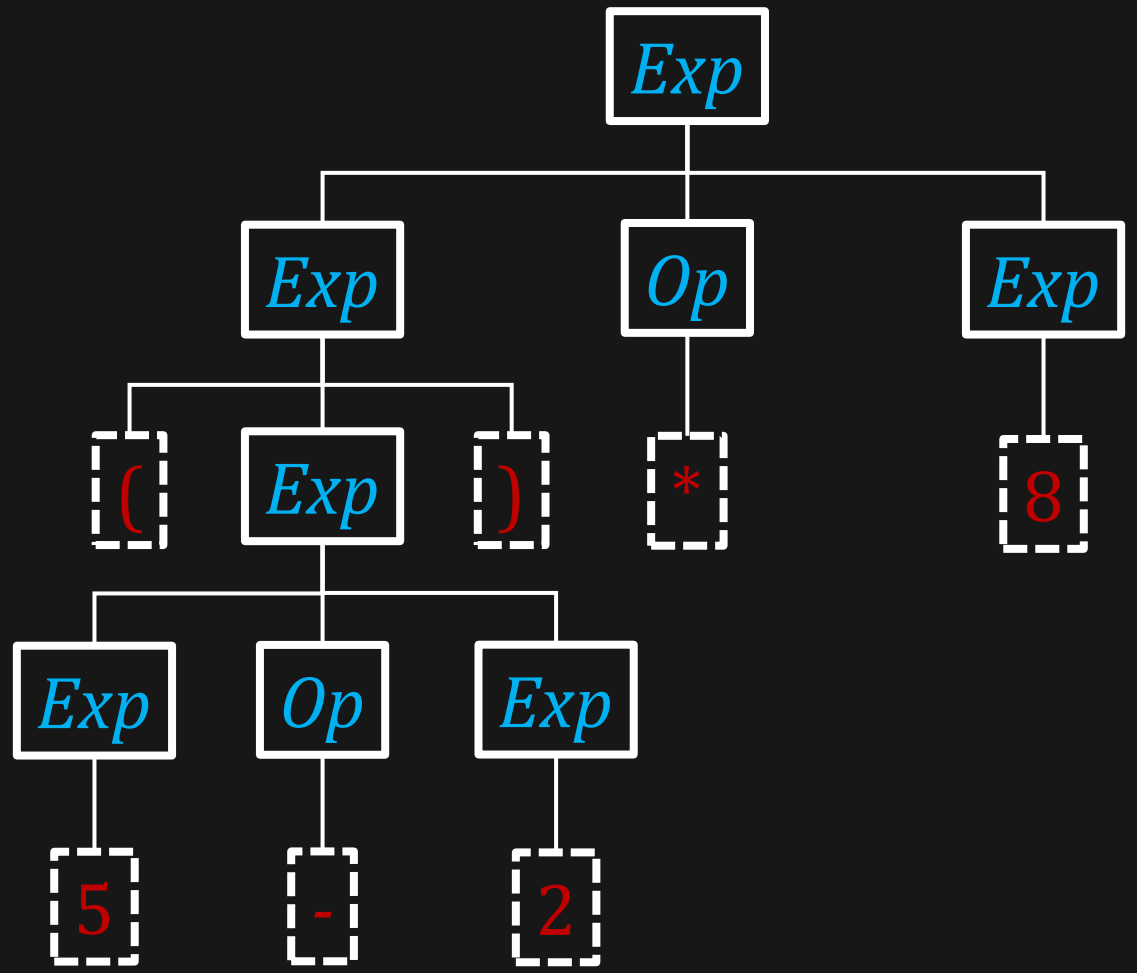
  $Exp \rightarrow (Exp)$

  $Exp \rightarrow \#$

  $Op \rightarrow + \mid - \mid *$

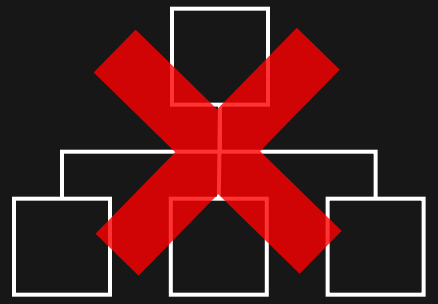
- Desarrolle la hilera:

  (5 – 2) * 8



¿PARA QUÉ EL ÁRBOL DE DERIVACIÓN?

- Es equivalente a la derivación
- Su existencia demuestra que la hilera (programa) es correcto sintácticamente
- ¿Si no hay forma de construir un árbol de derivación para una hilera?
  - Error de sintáxis
- Insumo de otras fases del compilador
  - Análisis semántico
  - Generación de código
- Un árbol muestra la interpretación que le daremos al código



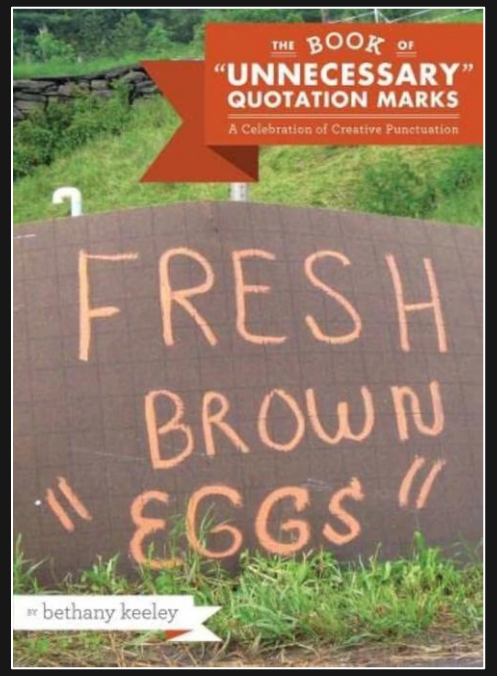
= Syntax Error

AMBIGÜEDAD

- Una información es ambigua si puede tener más de una interpretación
- Ocurre en lenguaje natural
  - Se murió el cerdo de Estaban
  - Juan vio a Pedro. Él usaba binoculares
- Con información conocida a priori o haciendo preguntas adicionales se puede romper la ambigüedad
- Problema equivalente para lenguajes formales

- Definición:

  Un CFG es ambiguo si existe al menos una hilera para la que haya al menos dos árboles de derivación más izquierda (o derecha) que sean diferentes





EJEMPLO DE AMBIGÜEDAD

- Considere la CFG:

  $Exp \rightarrow Exp\ Op\ Exp$

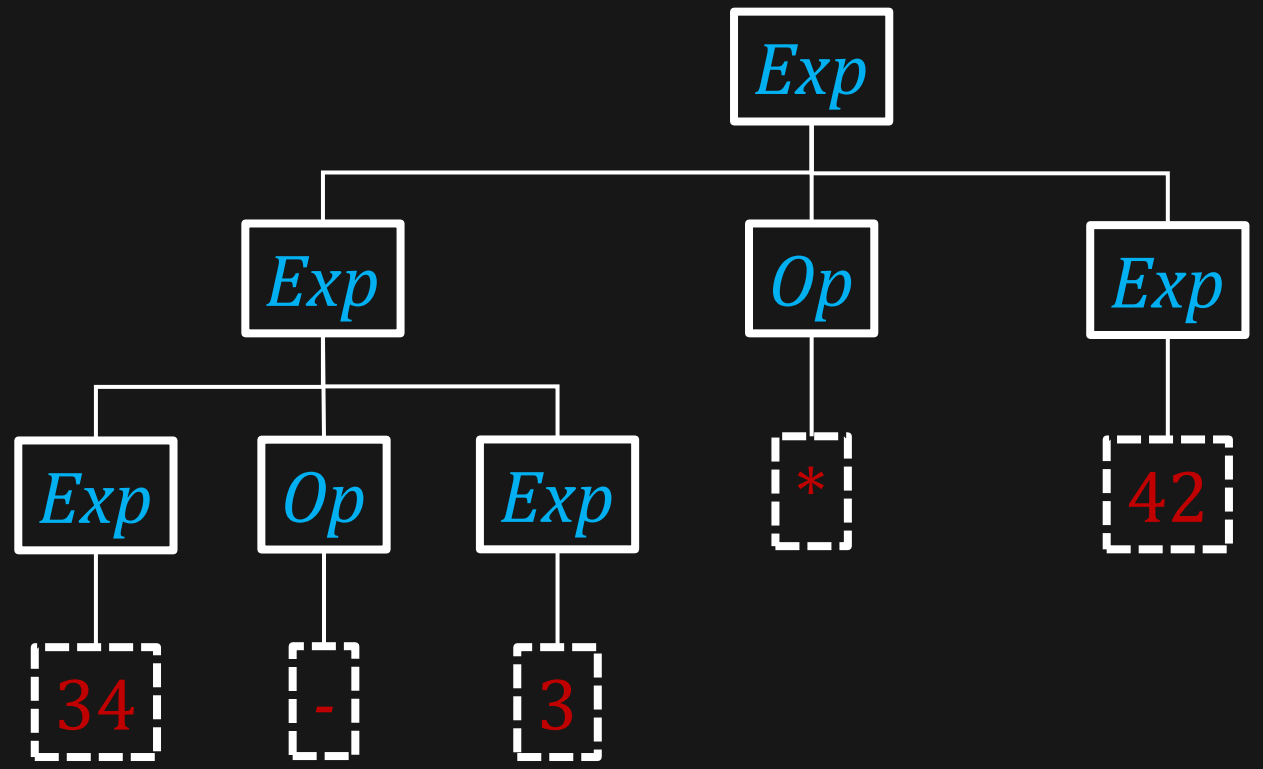
  $Exp \rightarrow (Exp)$

  $Exp \rightarrow \#$

  $Op \rightarrow + \mid - \mid *$

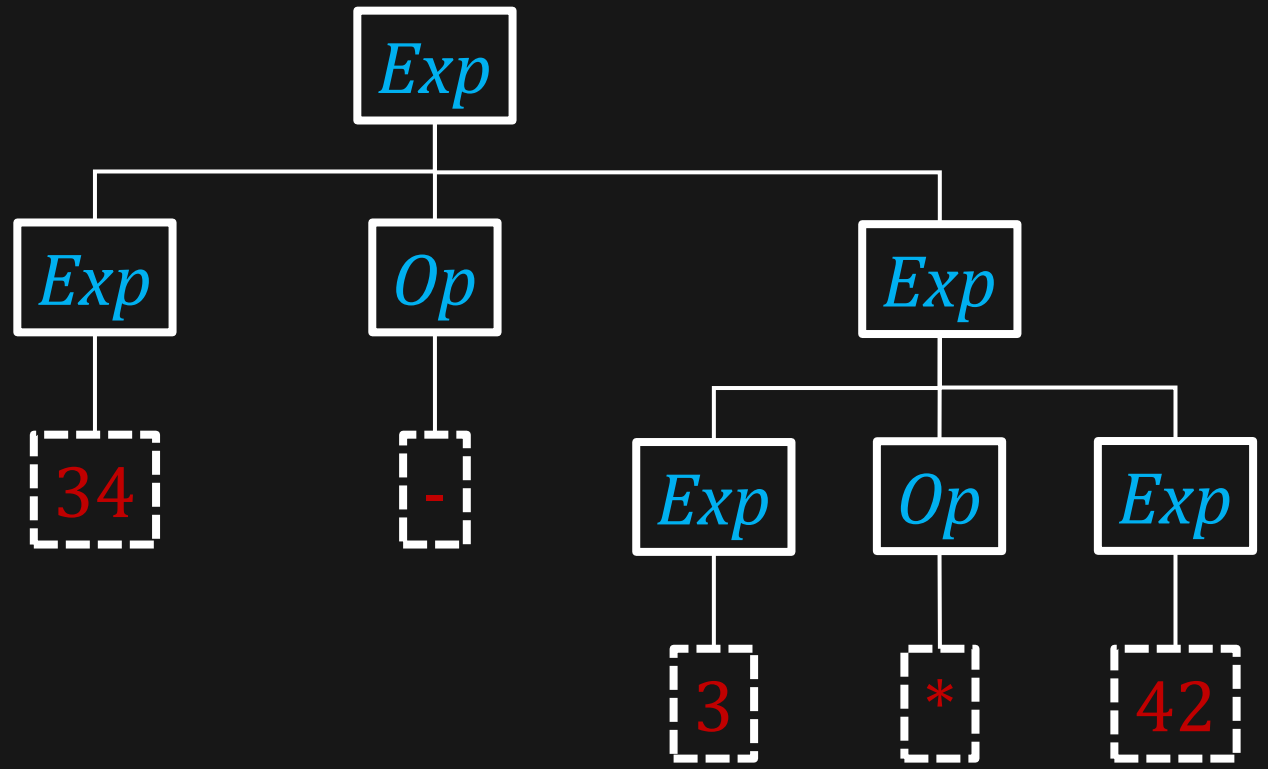
- Construya el árbol de derivación para la hilera:

  34 – 3 * 42



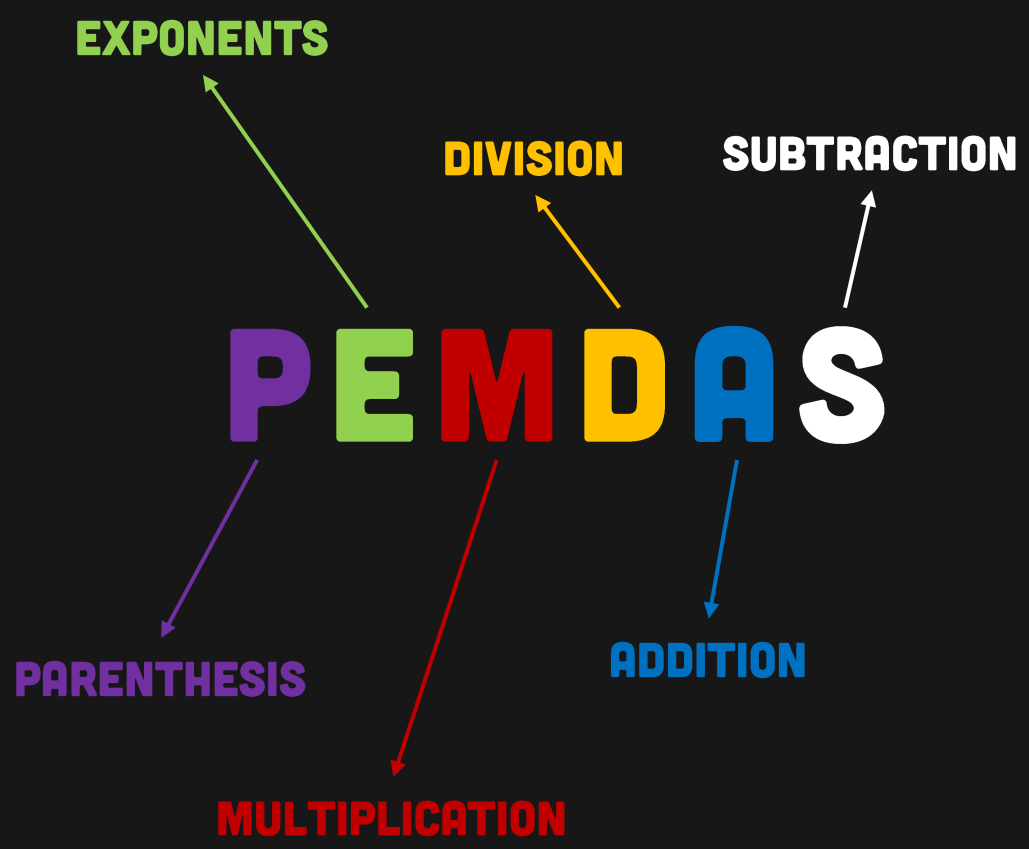
EJEMPLO DE AMBIGÜEDAD

- Pero… también se puede construir de esta otra forma
- Usando reglas de desambiguación se puede decidir cuál sería el correcto
- No hay un algoritmo que resuelva esto para cualquier lenguaje



PRECENDENCIA

- Se establece la precendencia entre dos operadores o construcciones
- La gramática tiene que reflejar esta precedencia
- Pueden haber parejas sin precedencia establecida
- Ejemplos típicos:
  - Multiplicación y división sobre sumas y restas
  - Exponenciación sobre multiplicación
  - Paréntesis sobre cualquier otro



EJEMPLO DE PRECEDENCIA

- Considere la CFG:

  $Exp \rightarrow Exp\ Addop\ Exp \mid Term$

  $Exp \rightarrow + \mid -$

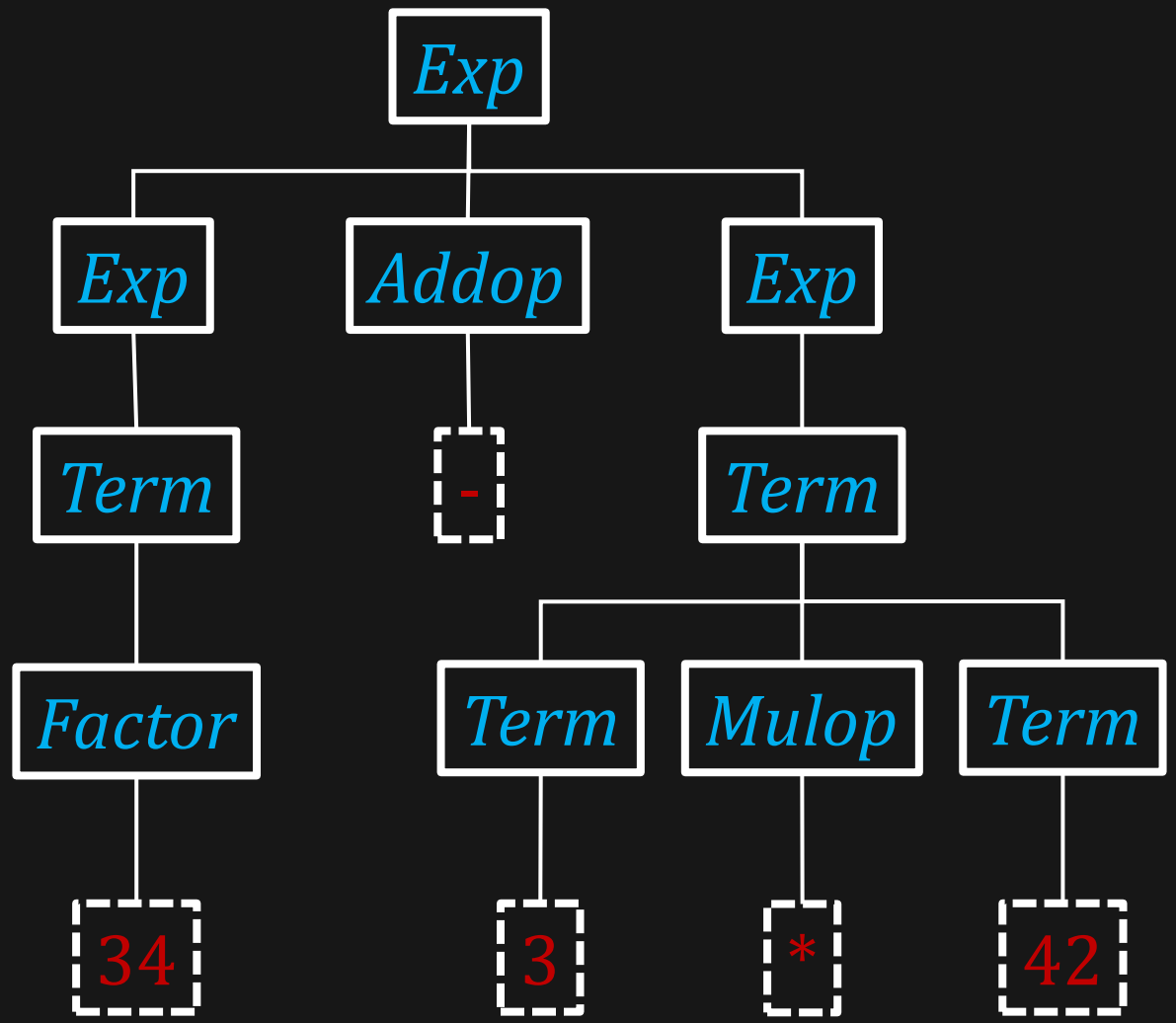
  $Term \rightarrow Term\ Mulop\ Term \mid Factor$

  $Factor \rightarrow (Exp) \mid \#$

- Trabajar la hilera:

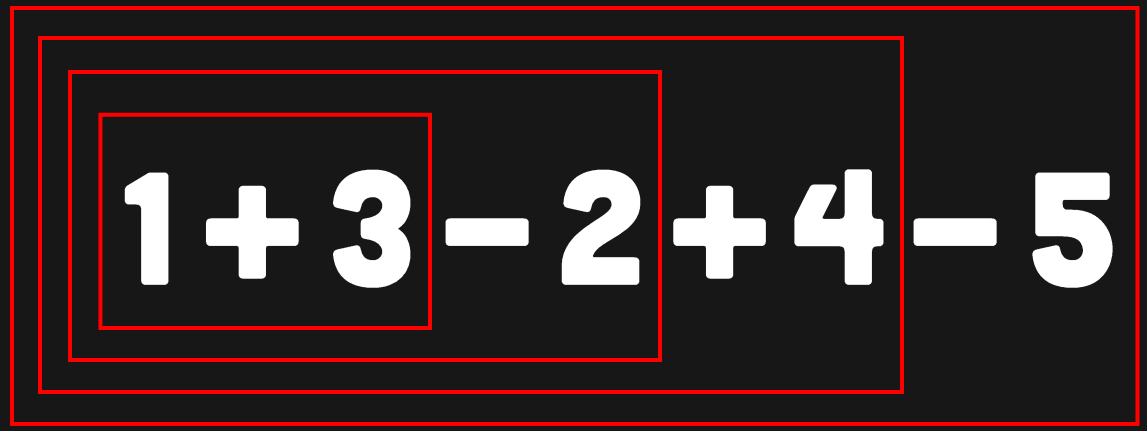
  34 – 3 * 42

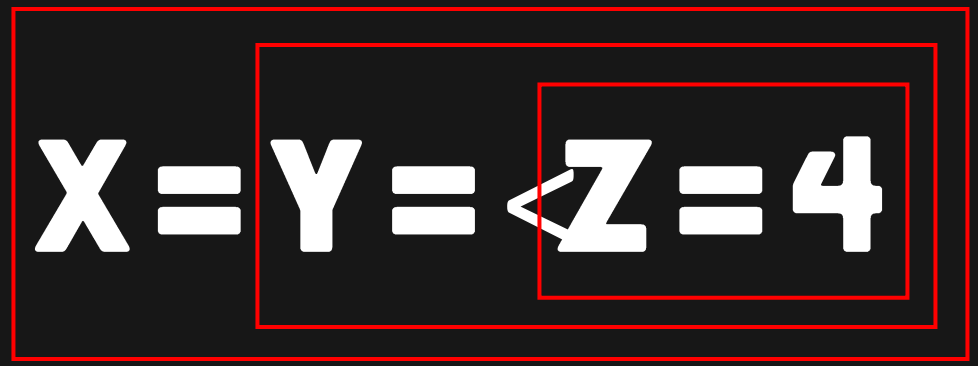
- Con este CFG se fuerza que la multiplicación precede la suma y resta



ASOCIATIVIDAD

- Dados dos operadores o construcciones iguales (sin precedencia) se debe establecer si son asociativos por la izquierda o por la derecha
- La gramática debe reflejar esta asociatividad
- Ejemplos típicos:
  - Multiplicación, división, resta, y suma son asociativos por la izquierda
  - Asignación en el lenguaje C es asociativo por la derecha





EJEMPLO DE ASOCIATIVIDAD

- Considere la CFG:

  $Exp \rightarrow Exp\ Addop\ Term \mid Term$

  $Exp \rightarrow + \mid -$

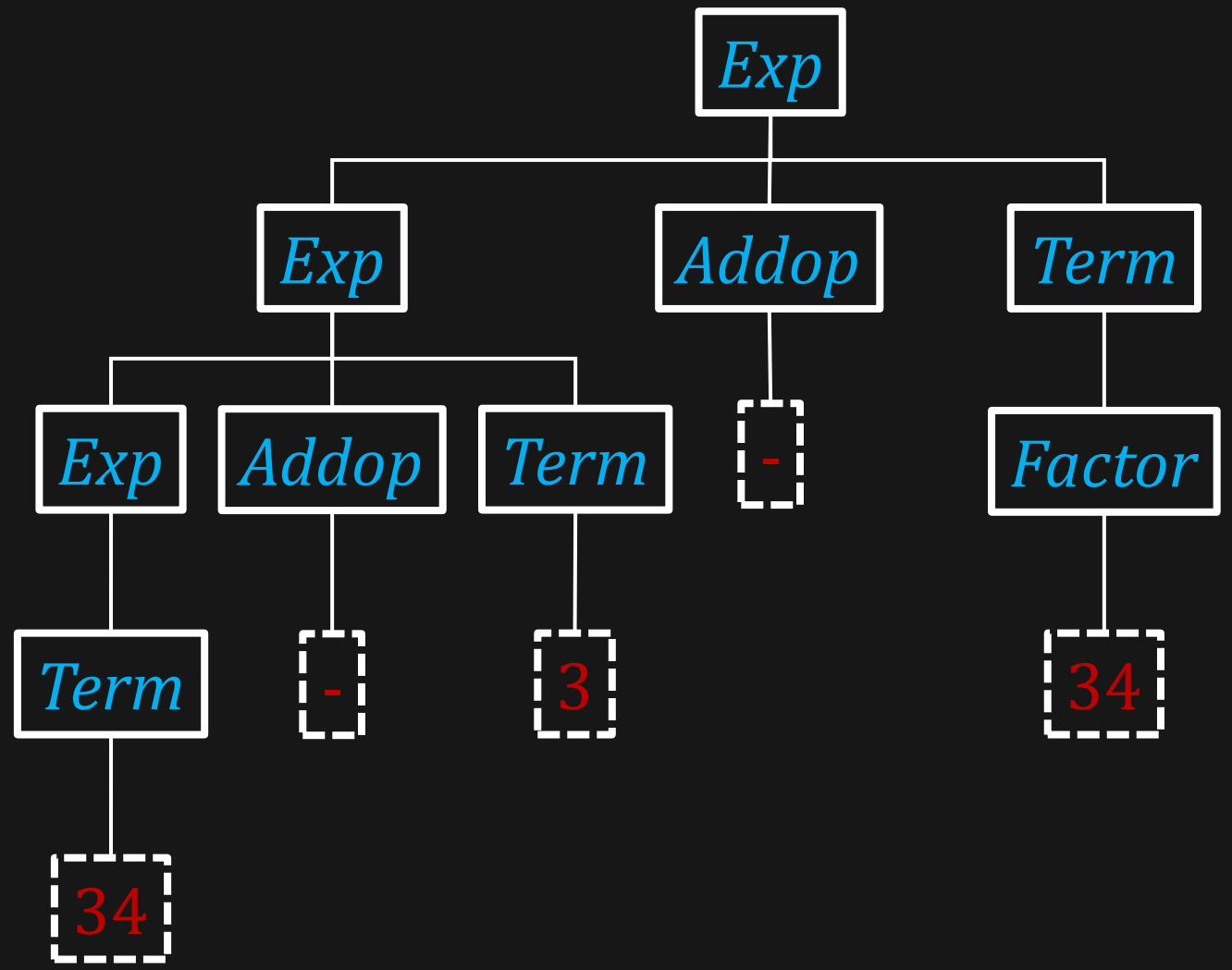
  $Term \rightarrow Term\ Mulop\ Factor \mid Factor$

  $Factor \rightarrow (Exp) \mid \#$

- Trabajar la hilera:

  34 – 3 – 42

- Con este CFG se fuerza que las operaciones se resuelvan de izquierda a derecha



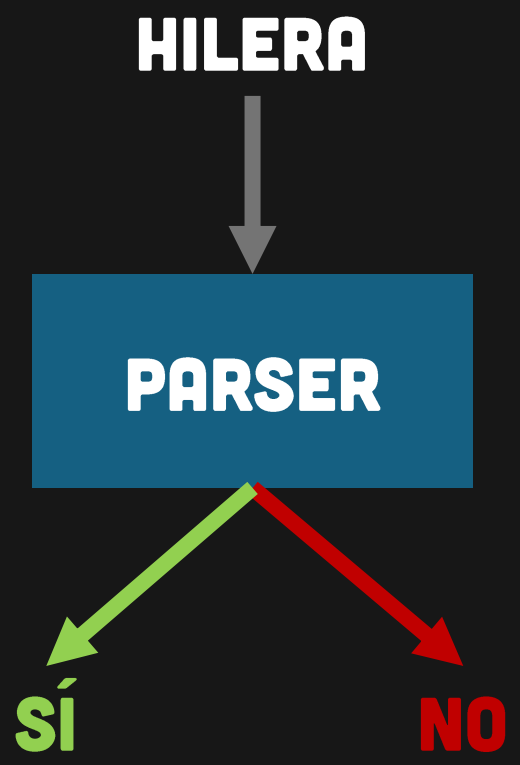
ES HORA DE CONOCER LA SIGUIENTE PARTE DEL COMPILADOR

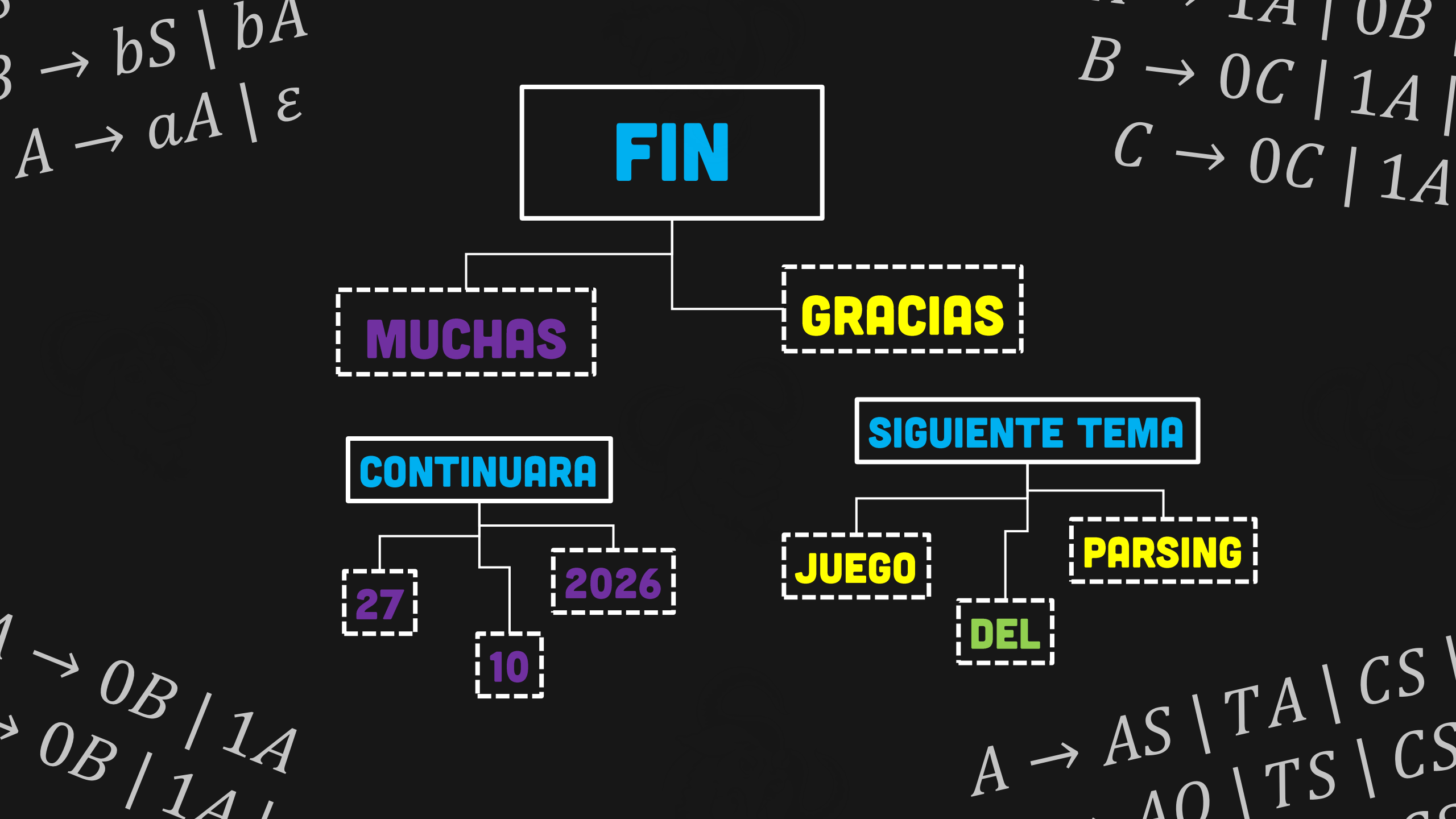
AL FINAL VAMOS A VER AL

EL PARSER

PARSING

- Una CFG genera un CFL
- Un Parser reconoce o detecta un CFL
- Se le prenta una hilera y dice si es parte del lenguaje




FIN
MUCHAS
GRACIAS
CONTINUARA
27
10
2026
SIGUIENTE TEMA
JUEGO
DEL
PARSING